

Project Proposal: Parallel Implementation of Breadth-First Search (BFS) Using Shared and Distributed Memory Models

1. Background

Graphs are widely used to represent relationships in systems such as social networks, transportation networks, and communication infrastructures. Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores vertices level by level from a source node. It is commonly applied in shortest path computation and connectivity analysis.

As graph sizes increase, serial BFS becomes inefficient because it executes sequentially and cannot fully utilize modern multi-core and distributed computing systems. Therefore, parallel computing techniques are required to improve performance and scalability.

2. Problem Statement

This project focuses on improving the performance of BFS on large-scale graphs using parallel programming models. A serial implementation will serve as the baseline, while shared-memory (OpenMP or POSIX Threads), distributed-memory (MPI), and hybrid implementations will be developed to enhance performance using OpenMP + CUDA.

The implementations will be evaluated by measuring execution time, computing speedup and efficiency, and verifying correctness through comparison with the serial output. The study will also analyse scalability, synchronization overhead, and communication cost under varying numbers of threads and processes.

3. Methodology

Synthetic undirected graphs with configurable sizes and edge densities will be generated for testing. Graphs will be stored using an adjacency list representation to ensure efficient memory usage and faster neighbour traversal.

A serial BFS implementation will first be developed to provide a correctness reference and performance baseline. Parallel implementations will then be constructed using a level-synchronous approach, where nodes at each BFS level are expanded concurrently.

In the shared-memory versions, **OpenMP** or **POSIX** Threads will be used to parallelize the frontier expansion across multiple threads with appropriate synchronization mechanisms. In the distributed-memory version, the graph will be partitioned among **MPI** processes, and frontier information will be exchanged between processes at each level. The hybrid version will combine OpenMP-based task parallelism with GPU management via CUDA.

4. Evaluation Matrices

Performance will be measured by varying the number of threads and MPI processes. Speedup and efficiency will be calculated, and correctness will be verified by comparing distance arrays from parallel versions with the serial implementation. Each experiment will be executed multiple times, and average execution times will be reported.